Tropical Monte Carlo

Comprehensive Reference Manual

A numerical integration pipeline for generalized Euler integrals using tropical geometry and Monte Carlo sampling.

Contents

1 Overview and Purpose **3**

2 Mathematical Background **3**
2.1 The Problem . . . 3
2.2 Tropical Geometry Approach . . . 3
2.3 Why This Helps . . . 4

3 File Structure **4**

4 Architecture: The 4-Stage Pipeline **4**
4.1 Stage 1: Tropical Fan Decomposition . . . 4
4.2 Stage 2: Symbolic Sector Processing . . . 5
4.3 Stage 3: Divergence Regulation . . . 5
4.4 Stage 4: C++ Code Generation and Execution . . . 6

5 Core File Reference **6**
5.1 `tropical_fan.wl` — Tropical Fan Decomposition (469 lines) . . . 6
5.2 `tropical_eval.wl` — Main Evaluation Package (2216 lines) . . . 7
5.2.1 Module 1: `ProcessSector` (Symbolic Sector Processing) . . . 7
5.2.2 Module 2: `ProcessDivergentSector` (Divergence Regulation) . . . 8
5.2.3 Module 2b: `IBPProcessSector` (IBP-Based Pole Extraction) . . . 8
5.2.4 Module 3: `MmaToC` and `GenerateCppMonteCarlo` (C++ Code Generation) 10
5.2.5 Module 4: `EvaluateTropicalMC` (Top-Level Driver) . . . 10

6 Data Structures **11**
6.1 `IntegrandSpec` (Mathematica Association) . . . 11
6.2 `SectorData` (from `ProcessSector`) . . . 12
6.3 `DivergentSectorData` (from `ProcessDivergentSector`) . . . 12
6.4 `IBPSectorData` (from `IBPProcessSector`) . . . 13

7 Key Algorithms **13**
7.1 Tropical (Newton Polytope) Decomposition . . . 13
7.2 Monomial Change of Variables . . . 13
7.3 Tropical Factoring . . . 14
7.4 Flattening . . . 14
7.5 Divergence Regulation (Tropical Subtraction) . . . 14
7.6 Log-Exp Polynomial Evaluation . . . 15
7.7 Welford Online Variance Estimation . . . 15
7.8 IBP-Based Pole Extraction . . . 15
7.9 Treatment of Complex Exponents . . . 16

8 Dependencies **17**
8.1 External Software . . . 17
8.2 Internal Dependencies . . . 17

9 Usage and Workflow **17**
9.1 Minimal Convergent Integral . . . 17
9.2 Kinematic Scan (Many Evaluation Points) . . . 18
9.3 Divergent Integral with Regulator . . . 18
9.4 Command-Line Execution . . . 19

10 Examples Directory **19**

11 Validation and Testing **20**
11.1 22-Test Suite (in `tropical_eval_examples.wl`) 20
11.2 FIESTA Cross-Checks (`CC/` Directory) 21

12 Limitations and Caveats **22**

13 Mathematical Properties and Why It Works **22**

1 Overview and Purpose

The **Tropical Monte Carlo** project is a numerical integration pipeline for evaluating generalized Euler integrals of the form:

$$I = \int_{[0,\infty)^n} dx_1 \cdots dx_n \prod_i x_i^{A_i} \prod_j P_j(\mathbf{x})^{B_j} \tag{1}$$

where:

- $P_j(\mathbf{x})$ are polynomials with real positive coefficients that may depend on kinematic parameters (e.g., masses, momenta in physics).
- A_i are the monomial exponents (may be complex, may depend on a dimensional regulator ε).
- B_j are the polynomial exponents (typically negative, may be complex, may depend on ε).

These integrals arise in quantum field theory (Feynman integrals), cosmological correlator calculations, and precision physics. They are often high-dimensional, and standard numerical quadrature fails for large n.

Key Innovation. The code uses **tropical geometry**—specifically the Newton polytope fan decomposition—to partition the integration domain into simplicial sectors. Within each sector, symbolic coordinate transformations and a "flattening" procedure render the integrand $\mathcal{O}(1)$ on the unit hypercube $[0,1]^n$, making uniform Monte Carlo sampling highly efficient.

A single C++ compilation handles thousands of kinematic points in parallel via OpenMP, since the fan decomposition depends only on the exponent structure, not on the polynomial coefficients.

2 Mathematical Background

2.1 The Problem

We want to compute:

$$I(\lambda) = \int_{[0,\infty)^n} \prod_i x_i^{A_i} \prod_j P_j(\mathbf{x};\lambda)^{B_j} \, d\mathbf{x} \tag{2}$$

for many values of kinematic parameters $\lambda = (\lambda_1, \lambda_2, \ldots)$.

Direct Monte Carlo on $[0,\infty)^n$ is impractical because:

- The integrand has singular behavior near coordinate hyperplanes ($x_i \to 0$).
- Different monomials dominate in different regions.
- Variance is enormous without importance sampling.

2.2 Tropical Geometry Approach

The tropical version of a polynomial $P(\mathbf{x}) = \sum_m c_m \mathbf{x}^m$ replaces multiplication with addition and addition with min:

$$\mathrm{trop}(P)(\mathbf{w}) = \min_m \{m \cdot \mathbf{w}\} \qquad (\mathbf{w} \in \mathbb{R}^n) \tag{3}$$

The set of $\mathbf{w}$ where the minimum is achieved by multiple monomials defines the *tropical variety*. The **Newton polytope** of P is the convex hull of the exponent vectors $\{m\}$. Its **normal fan** partitions $\mathbb{R}^n$ into cones, each corresponding to a region where one monomial dominates.

2.3 Why This Helps

In each cone (sector), the dominant monomial can be factored out symbolically. After a monomial change of variables aligned with the cone's rays, the remaining polynomial has no singular behavior—it equals $1 + (\text{bounded terms})$. A further "flattening" coordinate change absorbs the monomial singularity x^{A-1} into the measure, producing an $\mathcal{O}(1)$ integrand on $[0,1]^n$.

3 File Structure

```
TROPICAL_MONTE_CARLO/
|
|-- SUMMARY.txt                Project documentation
|-- tropical_fan.wl            Tropical fan computation (469 lines)
|-- tropical_eval.wl           Main evaluation package (3834 lines)
|
|-- EXAMPLES/
|   |-- tropical_eval_examples.wl 14 worked examples + 22-test suite
|   |-- tropical_fan_examples.wl Fan computation examples
|   |-- test_quick.wl          Minimal smoke test (Example 1)
|   |-- test_cpp.wl            C++ codegen & execution test
|   |-- test_divergent.wl      Divergent integral tests
|   |-- prompt_tropical_mc.txt Original development specification
|
|-- CC/                        Cross-checks vs FIESTA
|   |-- SUMMARY.txt            Comparison results (182 lines)
|   |-- bubblepp_test/         5D cosmological integral case study
|   |   |-- main_mc.cpp        Generated C++ (36 conv + 14 div sectors)
|   |   |-- g0_mc.cpp          G0 integral C++ source
|   |   |-- kin.txt            Kinematic data
|   |   |-- results_main.txt   MC results
|   |   |-- results_g0.txt     G0 results
|   |-- compare_*.wl           7 comparison scripts
|   |-- convergence_study*.wl  MC convergence analysis scripts
|   |-- diagnose_eps_eps.wl    eps/eps cross-term investigation
|
|-- INTERFILES/                Generated intermediate files
|   |-- .gitignore
|   |-- test_mc_generated.cpp  Example generated C++ source
|   |-- test_mc                Compiled binary
|   |-- test_kin_data.txt      Input kinematic file
|   |-- test_mc_results.txt    MC output results
|
|-- MANUAL/                    This manual
```

4 Architecture: The 4-Stage Pipeline

The code operates as a 4-stage pipeline:

4.1 Stage 1: Tropical Fan Decomposition

Source:

`tropical_fan.wl`

Input:

Newton polytope of $P_1 \cdot P_2 \cdots P_m$

Process:
Convex hull → normal fan → simplicial triangulation

Output:
Dual rays + simplicial cones (list of ray-index tuples)

External Tool:
Polymake (called via Perl API)

4.2 Stage 2: Symbolic Sector Processing

Source:
`tropical_eval.wl`, Module 1

Input:
Fan decomposition + `IntegrandSpec`

Process:
For each simplicial cone:

(a) Build ray matrix M, check $\det(M) \neq 0$
(b) Monomial change of variables $\mathbf{x} \to \mathbf{y}$ via M
(c) Tropical factoring (extract dominant monomials)
(d) Compute effective exponents a_{eff}
(e) Flattening substitution $\mathbf{y} \to \mathbf{y}'$ (absorb singularity)
(f) Divide exponents, compute prefactor

Output:
`SectorData` association per cone (13+ fields)

4.3 Stage 3: Divergence Regulation

Source:
`tropical_eval.wl`, Modules 2 and 2b

Input:
Sectors where $\mathrm{Re}(a_k(\varepsilon{=}0)) \leq 0$

Process:
Two methods are available:

Method A: Tropical Subtraction (Module 2, `ProcessDivergentSector`). Handles sectors with *one* divergent variable.

(a) Identify divergent variable k, compute c_k
(b) Construct G_0 integral ($(n{-}1)$-dimensional, pole coefficient)
(c) Construct G_1 integral ($G_0\times$ log-insertion terms)
(d) Construct Remainder (full minus simplified, convergent)
(e) Compute analytic pole $= 1/c_k$ symbolically

Output: `DivergentSectorData` association.

Method B: IBP Pole Extraction (Module 2b, `IBPProcessSector`). Handles sectors with one *or more* divergent variables (nested divergences) via integration by parts.

(a) Identify all divergent variables $y_{k_1}, y_{k_2}, \ldots$

(b) For each divergent variable, apply IBP to decompose the integral into a sum of monomial terms (each with shifted, convergent exponents) plus a boundary integral at $y_k = 1$

(c) Series-expand all terms in ε before flattening

(d) Flatten each term independently (different terms get different flattening transforms)

(e) Compute analytic pole structure: $P_{-d}/\varepsilon^d + \cdots + P_{-1}/\varepsilon + P_0$

Output: `IBPSectorData` association.

4.4 Stage 4: C++ Code Generation and Execution

Source:
`tropical_eval.wl`, Modules 3–4

Input:
All sector data + kinematic points

Process: (a) Translate Mathematica expressions to C++ strings
(b) Generate integrand functions (conv, g_0, g_1, remainder)
(c) Generate `main()` with file I/O, OpenMP, Welford statistics
(d) Compile with `g++ -std=c++17` (auto-detect OpenMP)
(e) Execute binary, read results from output file
(f) Combine MC results with analytic pole contributions

Output:
Results (mean ± error per kinematic point)

5 Core File Reference

5.1 `tropical_fan.wl` — Tropical Fan Decomposition (469 lines)

Purpose. Compute the tropical (Newton polytope) fan decomposition using the external tool Polymake.

Key Functions:

`PolytopeVertices[integrand, vars]`
Extracts the Newton polytope from the integrand expression. Collects all monomial exponent vectors from the product of polynomials. Returns a list of integer vectors (lattice points).

`convexHullVertices[points]`
Calls Polymake to compute the convex hull of a set of points. Generates a Perl script that invokes Polymake's polytope library. Returns the vertices of the convex hull.

`translateToOriginInteger[vertices]`
Shifts the polytope so that the origin is an interior point. Uses an interior lattice point for the translation. Important for numerical stability of the fan computation.

`ComputeDecomposition[vertices]`
: Master function: triangulates the polytope into simplicial cones. Calls Polymake for the dual fan and simplicial subdivision. Returns `{dualVertices, simplexList}` where each simplex is a tuple of ray indices defining one simplicial cone.

`ComputeDecompositiony[vertices]`
: Variant that intersects the fan with the positive orthant. Used when the integration domain requires special handling.

Polymake Interface. The code auto-detects Polymake at `/opt/homebrew`, `/usr/local`, or `/usr`. It generates temporary Perl scripts passed to Polymake and parses the text output back into Mathematica lists.

Typical Output. For a 2D polynomial with 4 terms:

```
dualVertices = {{1,0}, {0,1}, {-1,0}, {0,-1}, {1,1}, ...}
simplexList = {{1,2}, {2,3}, {3,4}, {4,5}, ...}
```

5.2 `tropical_eval.wl` — Main Evaluation Package (2216 lines)

5.2.1 Module 1: `ProcessSector` (Symbolic Sector Processing)

Purpose. Perform the symbolic coordinate transformation for one simplicial cone, converting the integrand into a form suitable for Monte Carlo sampling on $[0,1]^n$.

Function Signature:

```
ProcessSector[integrandSpec, dualVertices, simplex, coneIndex]
```

Algorithm:

1. **Extract rays and build matrix.** Take the n rays for this cone and build $M = -\text{Transpose}[\{\rho_1, \ldots, \rho_n\}]$. The change of variables is $x_i = \prod_j y_j^{M_{ij}}$.
2. **Check non-degeneracy.** Verify $\det(M) \neq 0$. Degenerate cones are skipped.
3. **Transform polynomials.** Substitute $\mathbf{x} \to \mathbf{y}^M$ in all polynomials: each monomial $c \cdot \mathbf{x}^\alpha$ becomes $c \cdot \mathbf{y}^{\alpha \cdot M}$.
4. **Tropical factoring.** For each polynomial $P_k(\mathbf{y})$, find
$$d_{k,j} = \min_m(\text{exponent of } y_j \text{ in monomial } m \text{ of } P_k) \tag{4}$$
Factor out $\mathbf{y}^{d_k}$ to get $Q_k(\mathbf{y})$ with all non-negative exponents. Q_k always contains a constant term (the dominant monomial).
5. **Compute effective exponents:**
$$a_{\text{eff},i} = \text{rawA}_i + \sum_k B_k \cdot d_{k,i} \tag{5}$$
6. **Check convergence.** If $\text{Re}(a_{\text{eff},i}) > 0$ for all i, the sector is convergent. Otherwise, flag as divergent.
7. **Flatten.** Substitute $y_i \to (y_i')^{1/a_{\text{eff},i}}$. New exponents become $e_{m,i}/a_{\text{eff},i}$.
8. **Compute prefactor:**
$$\text{prefactor} = \frac{|\det(M)|}{\prod_i a_{\text{eff},i}} \tag{6}$$

Output. A `SectorData` association with 13+ fields (see Section 6.2).

5.2.2 Module 2: `ProcessDivergentSector` (Divergence Regulation)

Purpose. Handle sectors where one integration variable diverges as $1/\varepsilon$, extracting the pole analytically and leaving convergent numerical integrals.

Prerequisite: `IdentifyDivergences[sectorData, ε]`

- Expands $a_k(\varepsilon) = c_k\varepsilon + \mathcal{O}(\varepsilon^2)$.
- Identifies the divergent variable k (with $c_k > 0$, $a_k(0) \leq 0$).
- Currently handles at most one divergent variable per sector.

Decomposition (Tropical Subtraction). The integral for a divergent sector has the structure:

$$I_{\text{sector}} = \int y_k^{c_k\varepsilon - 1} \cdot f(\mathbf{y})\, d\mathbf{y} \tag{7}$$

which has a $1/(c_k\varepsilon)$ pole from the $y_k \to 0$ region. The decomposition is:

$$I_{\text{sector}} = \frac{G_0}{c_k\varepsilon} + \frac{G_1}{c_k} + R \tag{8}$$

where:

G_0**:** $(n-1)$-dimensional integral obtained by setting $y_k = 0$. This is the pole coefficient.

G_1**:** G_0 integrand multiplied by a log-insertion sum:

$$\sum_i \frac{a_i^{(1)}}{a_i^{(0)}} \log(y_i') + \sum_j B_j^{(1)} \log(Q_j|_{y_k=0}) \tag{9}$$

where $a^{(0)}, a^{(1)}$ are the ε^0 and ε^1 coefficients.

R **(Remainder):** Integral of $[\text{full} - \text{simplified}] \times y_k^{a_{0,k}-1}$, which is convergent because the divergent piece has been subtracted.

The analytic pole $1/c_k$ is computed symbolically in Mathematica. G_0, G_1, and R are evaluated numerically via C++ Monte Carlo.

5.2.3 Module 2b: `IBPProcessSector` (IBP-Based Pole Extraction)

Purpose. Handle divergent sectors—including those with *multiple* divergent variables (nested divergences)—using integration by parts. This module is fully additive: it does not modify any existing functions. The tropical subtraction scheme (Module 2) is retained alongside it.

Mathematical Idea. For a divergent variable y_k with effective exponent $a_k = c_k\varepsilon + \mathcal{O}(\varepsilon^2)$, the IBP identity on $[0, 1]$ gives:

$$\int_0^1 y_k^{a_k-1} f(\mathbf{y})\, dy_k = \frac{1}{a_k}\Big[f(\mathbf{y})\big|_{y_k=1} - \int_0^1 y_k^{a_k}\, \partial_{y_k} f(\mathbf{y})\, dy_k\Big] \tag{10}$$

The derivative ∂_{y_k} acts on the polynomial product $\prod_j Q_j^{B_j}$:

$$\partial_{y_k} \prod_j Q_j^{B_j} = \sum_j B_j\, Q_j^{B_j-1}\, (\partial_{y_k} Q_j) \prod_{l\neq j} Q_l^{B_l} \tag{11}$$

Each monomial of $\partial_{y_k} Q_j$ produces a separate integral term with shifted exponents. Crucially, the new effective exponent for y_k is $a_k + e_{m,k} \geq 1$ at $\varepsilon = 0$ (since $e_{m,k} \geq 1$ for monomials contributing to the derivative), so the divergence in y_k is resolved.

Boundary Term. On the domain $[0,1]^n$, the IBP produces a non-vanishing boundary contribution at $y_k = 1$:

$$B_{\text{boundary}} = \int_{[0,1]^{n-1}} f(\mathbf{y})\big|_{y_k=1}\, d\mathbf{y}_{\neq k} \tag{12}$$

This is an $(n-1)$-dimensional convergent integral with the same structure as G_0 (but evaluating Q_j at $y_k = 1$ instead of $y_k = 0$). The boundary at $y_k = 0$ vanishes for $\mathrm{Re}(a_k) > 0$.

Full Formula (Single Divergence).

$$I_{\text{sector}} = \frac{1}{a_k}\Big[B_{\text{boundary}} - \sum_t \text{coeff}_t\, I_t\Big] \tag{13}$$

where each I_t is a convergent n-dimensional integral from the monomial decomposition of the derivative, and $\text{coeff}_t = B_j \cdot c_m \cdot e_{m,k}$.

ε-Expansion and Pole Structure. The prefactor $1/a_k = 1/(c_k\varepsilon)(1 - r_k\varepsilon + \cdots)$ produces the $1/\varepsilon$ pole:

$$\text{Pole:} \quad \frac{B^{(0)} - S_0}{c_k\varepsilon} \tag{14}$$

$$\text{Finite:} \quad \frac{(B^{(1)} - S_1) - r_k(B^{(0)} - S_0)}{c_k} \tag{15}$$

where $S_0 = \sum_t \text{coeff}_t^{(0)} I_{t,0}$, $S_1 = \sum_t(\text{coeff}_t^{(0)} I_{t,1} + \text{coeff}_t^{(1)} I_{t,0})$, and $r_k = a_k^{(2)}/c_k$.

Nested Divergences. When multiple variables $y_{k_1}, y_{k_2}, \ldots$ are divergent, the IBP is applied iteratively. Each step resolves one variable and may split terms further. The prefactor becomes $\prod_m(-1/a_{k_m})$, producing poles up to $1/\varepsilon^d$ for d divergent variables.

Key Functions:

`IBPReduceSector[sectorData, eps]` Symbolic IBP decomposition: identifies all divergent variables and iteratively applies IBP to produce a list of convergent terms.

`IBPProcessSector[sectorData, integrandSpec]` Full pipeline: IBP reduction + ε-expansion + per-term flattening + boundary construction. Returns an `IBPSectorData` association ready for C++ codegen.

`IBPCheckBoundary[sectorData, integrandSpec, nPoints]` Numerically verifies that IBP boundary terms are well-behaved.

`ValidateIBP[ibpSD, sectorData, integrandSpec, kin, eps]` Cross-checks the IBP formula $(1/a_k)[B - \sum_t \text{coeff}_t I_t]$ against direct `NIntegrate` at finite ε.

`GenerateCppMonteCarloIBP[conv, ibp, spec, file]` Generates C++ code with per-function output for IBP integrands.

`EvaluateTropicalMCIBP[spec, fanData, kin, opts]` Top-level driver using IBP for all divergent sectors. Returns pole coefficients and finite parts.

5.2.4 Module 3: `MmaToC` and `GenerateCppMonteCarlo` (C++ Code Generation)

`MmaToC[expr, kinematicSymbols].` Recursive pattern-matching converter from Mathematica to C++:

Mathematica	**C++**
Integer n	`n.0`
Rational p/q	`(p.0/q.0)`
`Complex[a,b]`	`cx(a, b)`
`Power[x, n]`	`std::pow(x, n)`
`Log[x]`	`std::log(x)`
`Plus[a, b, ...]`	`(a + b + ...)`
`Times[a, b, ...]`	`(a * b * ...)`
Kinematic symbol	`params[i]`
Variable `y[j]`	`y[j-1]`

Polynomial evaluation uses the **log-exp trick**:

$$c \cdot y_1^{\alpha_1} \cdot y_2^{\alpha_2} = c \cdot \exp\big(\alpha_1 \log(y_1) + \alpha_2 \log(y_2)\big) \tag{16}$$

`GenerateCppMonteCarlo[...].` Generates a complete C++ source file with:

- **4 types of integrand functions** per relevant sector:

  `integrand_conv_N`: Convergent sector—returns prefactor $\times \prod_j Q_j(\mathbf{y})^{B_j}$.

  `integrand_g0_N`: Divergent sector G_0—$(n-1)$-dim with simplified polynomials.

  `integrand_g1_N`: Divergent sector G_1—$G_0\times$ log-insertion sum.

  `integrand_rem_N`: Divergent sector Remainder—prefactor $\times\, y_k^{a_{0,k}-1} \times (\text{full} - \text{simplified})$.

- **Function table**: Array of `{function_pointer, dimension, type_flag}` for dispatch.

- **`main()` function**: File I/O for kinematic data, OpenMP parallelization over kinematic points, Welford online algorithm for mean and variance, optional debug mode with NaN detection.

- **Output format**: One line per kinematic point with columns: `real_part` `imag_part` `error_real` `er`

Compilation:

```
g++ -O2 -std=c++17 -fopenmp output.cpp -o output
```

Auto-retries without `-fopenmp` if linking fails (e.g., on macOS with clang).

5.2.5 Module 4: `EvaluateTropicalMC` (Top-Level Driver)

Purpose. Orchestrate the entire pipeline from `IntegrandSpec` to final numerical results.

Function Signature:

```
EvaluateTropicalMC[integrandSpec, fanData, kinematicPoints, opts...]
```

Algorithm:

1. Validate `IntegrandSpec` (check all required fields).
2. `ProcessSector` for all simplicial cones in the fan.
3. (Optional) `ValidateDecomposition`: cross-check one kinematic point against Mathematica's `NIntegrate`.
4. Separate sectors into convergent and divergent lists.
5. `ProcessDivergentSector` for each divergent sector.
6. Substitute $\varepsilon \to$ `epsValue` in all symbolic expressions.
7. Write kinematic data to temporary file.
8. `GenerateCppMonteCarlo` (write C++ source).
9. Compile C++ (debug build first if requested, then release).
10. Execute binary, read results from output file.
11. Combine MC results with analytic pole contributions:

$$\text{total} = \sum_{\text{conv}} I_{\text{conv}} + \sum_{\text{div}} \left[\frac{G_0}{c_k \varepsilon} + \frac{G_1}{c_k} + R \right] \tag{17}$$

12. Print error summary, optionally validate against `NIntegrate`.

Options:

Option	Default	Description
`EpsilonValue`	0.01	Value of regulator ε
`NSamples`	1 000 000	MC samples per sector
`DebugMode`	`False`	Enable debug C++ build
`ValidateFirst`	`False`	Run `NIntegrate` cross-check
`OutputFile`	(auto)	Path for C++ source

6 Data Structures

6.1 `IntegrandSpec` (Mathematica Association)

Key	Description
`"Polynomials"`	$\{P_1, P_2, \ldots\}$ — symbolic polynomial expressions. Coefficients may depend on kinematic symbols.
`"MonomialExponents"`	$\{A_1, \ldots, A_n\}$ — one per integration variable. May be numeric, symbolic in ε, or complex.
`"PolynomialExponents"`	$\{B_1, \ldots, B_m\}$ — one per polynomial. Typically negative. May depend on ε.
`"Variables"`	$\{x_1, \ldots, x_n\}$ — integration variable symbols.
`"KinematicSymbols"`	$\{\lambda, \mu, \ldots\}$ or $\{\}$ — parameters varying across evaluation points.
`"RegulatorSymbol"`	ε or `None` — dimensional regulator.

6.2 `SectorData` (from `ProcessSector`)

Key	Description
`"ConeIndex"`	Integer label for this simplicial cone.
`"RayMatrix"`	M $(n \times n)$. Defines the change of variables $\mathbf{x} = \mathbf{y}^M$.
`"DetM"`	$\det(M)$. $\lvert\det(M)\rvert$ appears in the Jacobian.
`"SelectedRays"`	$\{\rho_1, \ldots, \rho_n\}$ — ray vectors defining this cone.
`"RawExponents"`	Exponents after the M-transformation, before tropical factoring.
`"NewExponents"`	Effective exponents: $a_{\text{eff},i} = \text{rawA}_i + \sum_k B_k d_{k,i}$.
`"MinExponents"`	$\{d_{k,j}\}$ per polynomial. Minimum exponent of y_j across monomials of P_k.
`"TransformedPolys"`	Monomials in $\mathbf{y}$ (exponents may be negative).
`"ClearedPolys"`	After tropical factoring. All exponents ≥ 0.
`"FlattenedPolys"`	After flattening. Exponents are $e_{m,i}/a_{\text{eff},i}$.
`"Prefactor"`	$\lvert\det(M)\rvert / \prod_i a_{\text{eff},i}$. May depend on ε.
`"IsDivergent"`	`True` or `False`.
`"DivergentVariable"`	Index k (1-based) or 0 if convergent.
`"Dimension"`	n, number of integration variables.
`"PolynomialExponents"`	$\{B_1, \ldots\}$, copied from `IntegrandSpec`.

6.3 `DivergentSectorData` (from `ProcessDivergentSector`)

Extends `SectorData` with:

Key	Description
`"G0FlatPolys"`	$(n-1)$-dim flattened polynomials with y_k set to 0.
`"G0Prefactor"`	Prefactor for the G_0 integral.
`"G0Dimension"`	$n - 1$.
`"G1LogInsertions"`	Association with `"VariableTerms"` $(a_i^{(1)}/a_i^{(0)})$ and `"PolynomialTerms"` $(B_j^{(1)})$.
`"Remainder"`	Association: `"FullPolys"`, `"SimplifiedPolys"`, `"Prefactor"`, `"DivVarExponent"`.
`"AnalyticPole"`	$1/c_k$ (symbolic).
`"ck"`	Leading coefficient of $a_k(\varepsilon)$.

6.4 `IBPSectorData` (from `IBPProcessSector`)

Key	Description
`"ConeIndex"`	Integer label for this sector.
`"Method"`	`"IBP"` (distinguishes from subtraction).
`"DivergentVariables"`	List of divergent variable indices.
`"NDivergent"`	Number of divergent variables.
`"ck"`	Leading coefficient c_k of $a_k(\varepsilon)$.
`"rk"`	Second-order correction $a_k^{(2)}/c_k$.
`"BoundaryData"`	Association for the $y_k = 1$ boundary integral: `"FlatPolys"`, `"Prefactor"`, `"Dimension"` $(n-1)$, `"PolyExponents"`, `"LogInsertions"`.
`"IBPTerms"`	List of term associations, each with: `"FlatPolys"`, `"Prefactor"`, `"Dimension"` (n), `"PolyExponents"`, `"Coeff0"`, `"Coeff1"`, `"LogInsertions"`, `"Alpha0"`, `"Alpha1"`.
`"NTerms"`	Number of IBP terms.
`"IBPPrefactors"`	List of $-1/a_{k_m}$ factors (one per IBP step).
`"AnalyticPole"`	$1/c_k$ (symbolic).

7 Key Algorithms

7.1 Tropical (Newton Polytope) Decomposition

Given polynomials $P_1, \ldots, P_m$, form the product $P = P_1 \cdots P_m$.

The Newton polytope $\mathrm{NP}(P)$ is the convex hull of the exponent vectors $\{\alpha : \mathbf{x}^\alpha \text{ appears in } P\}$.

The **normal fan** of $\mathrm{NP}(P)$ partitions $\mathbb{R}^n$ into polyhedral cones. Each full-dimensional cone C_σ corresponds to a vertex σ of $\mathrm{NP}(P)$. Points $\mathbf{w} \in C_\sigma$ satisfy: the linear functional $\mathbf{w} \cdot \alpha$ is minimized (over $\mathrm{NP}(P)$) precisely at σ.

The code:

1. Computes the Newton polytope vertices via Polymake.
2. Computes the normal fan (dual vertices = ray generators).
3. Triangulates non-simplicial cones into simplicial ones.
4. Returns $\{\texttt{dualVertices}, \texttt{simplexList}\}$.

7.2 Monomial Change of Variables

For a simplicial cone with rays $\{\rho_1, \ldots, \rho_n\}$, define:

$$M = -\mathrm{Transpose}[\rho_1, \ldots, \rho_n] \qquad (n \times n \text{ integer matrix}) \tag{18}$$

The change of variables is:

$$x_i = \prod_j y_j^{M_{ij}} \tag{19}$$

The Jacobian is:

$$dx_1 \cdots dx_n = |\det(M)| \cdot \prod_i y_i^{(\sum_j M_{ji})-1} \, dy_1 \cdots dy_n \tag{20}$$

The "raw exponents" after combining with the monomial factor are:

$$\mathrm{rawA}_j = -1 + \sum_i (A_i + 1) \cdot M_{ij} \tag{21}$$

7.3 Tropical Factoring

After the M-transformation, each polynomial $P_k(\mathbf{y})$ is a sum of monomials where some exponents may be negative. Define:

$$d_{k,j} = \min_m \{e_{m,j}\} \qquad \text{(min over all monomials } m \text{ of } P_k) \tag{22}$$

Then:

$$P_k(\mathbf{y}) = \mathbf{y}^{d_k} \cdot Q_k(\mathbf{y}) \tag{23}$$

where Q_k has all non-negative exponents and contains at least one constant term (the dominant monomial). This ensures $Q_k(\mathbf{y}) \geq \text{const} > 0$ for $\mathbf{y} \in [0,1]^n$.

The effective exponents become:

$$a_{\text{eff},i} = \text{rawA}_i + \sum_k B_k \cdot d_{k,i} \tag{24}$$

7.4 Flattening

After tropical factoring, the integrand on $[0,\infty)^n$ has the form:

$$\prod_i y_i^{a_{\text{eff},i}-1} \cdot \prod_j Q_j(\mathbf{y})^{B_j} \cdot |\det(M)| \tag{25}$$

The substitution $y_i = (y_i')^{1/a_{\text{eff},i}}$ yields:

$$y_i^{a_{\text{eff},i}-1}\, dy_i = \frac{1}{a_{\text{eff},i}}\, dy_i' \tag{26}$$

The y^{a-1} factor is completely absorbed. The integrand becomes:

$$\frac{|\det(M)|}{\prod_i a_{\text{eff},i}} \cdot \prod_j Q_j(\mathbf{y}'^{\,1/a_{\text{eff}}})^{B_j} \tag{27}$$

which is $\mathcal{O}(1)$ on $[0,1]^n$, ideal for uniform Monte Carlo sampling.

7.5 Divergence Regulation (Tropical Subtraction)

When $\text{Re}(a_{\text{eff},k}) \leq 0$ for $\varepsilon = 0$, write $a_k(\varepsilon) = c_k\varepsilon + \mathcal{O}(\varepsilon^2)$ with $c_k > 0$. The integral over y_k diverges as $1/(c_k\varepsilon)$.

Subtraction scheme:

$$I_{\text{sector}} = \int y_k^{c_k\varepsilon-1}[f(\mathbf{y}) - f(\mathbf{y}|_{y_k=0})]\, d\mathbf{y} + \int y_k^{c_k\varepsilon-1} f(\mathbf{y}|_{y_k=0})\, d\mathbf{y} \tag{28}$$

The first integral is convergent (the difference vanishes at $y_k = 0$). The second factors because $f(\mathbf{y}|_{y_k=0})$ is independent of y_k:

$$\int_0^1 y_k^{c_k\varepsilon-1} dy_k = \frac{1}{c_k\varepsilon} \tag{29}$$

This gives:

$$\text{Pole:} \quad \frac{G_0}{c_k\varepsilon} \tag{30}$$

$$\text{Finite:} \quad \frac{G_1}{c_k} \quad \text{(from } \varepsilon\text{-expansion, log insertions)} \tag{31}$$

$$\text{Remainder:} \quad R \quad \text{(convergent difference integral)} \tag{32}$$

7.6 Log-Exp Polynomial Evaluation

In the C++ code, monomials with fractional or complex exponents are evaluated using:

$$c \cdot y_1^{\alpha_1} \cdots y_n^{\alpha_n} = c \cdot \exp\big(\alpha_1 \log y_1 + \cdots + \alpha_n \log y_n\big) \tag{33}$$

This is numerically stable because:

- For $y_i \in (0, 1]$: $\log(y_i) \in (-\infty, 0]$, smooth and monotone.
- As $y_i \to 0^+$: $\log(y_i) \to -\infty$, but $\exp(\cdots) \to 0$ gracefully.
- No branch cuts or sign issues for real $y_i > 0$.
- Works for complex α (the result is complex-valued).
- Avoids platform-dependent behavior of `pow(y, fractional)`.

7.7 Welford Online Variance Estimation

For each integrand function and kinematic point, Monte Carlo estimates the mean and variance using Welford's online algorithm:

```
mean = 0, M2 = 0
for k = 1, 2, ..., N:
    v = integrand(random_point)
    delta = v - mean
    mean += delta / k
    delta2 = v - mean
    M2 += delta * delta2

variance = M2 / (N - 1)
std_error = sqrt(variance / N)
```

Properties:

- Numerically stable (no catastrophic cancellation).
- $\mathcal{O}(1)$ memory per integrand (just mean, M2, count).
- Real and imaginary parts tracked independently.
- Sector sums are independent: $\mathrm{Var}(\text{total}) = \sum \mathrm{Var}(\text{sector})$.

7.8 IBP-Based Pole Extraction

An alternative to the tropical subtraction scheme, the IBP method resolves divergences by integration by parts. For a divergent variable y_k with $a_k = c_k\varepsilon + \mathcal{O}(\varepsilon^2)$:

Step 1: IBP identity on $[0, 1]$.

$$\int_0^1 y_k^{a_k-1} f(\mathbf{y})\, dy_k = \frac{1}{a_k}\Big[f\big|_{y_k=1} - \int_0^1 y_k^{a_k} \partial_{y_k} f\, dy_k\Big] \tag{34}$$

Step 2: Monomial decomposition of derivative. $\partial_{y_k} Q_j = \sum_{m:\, e_{m,k}>0} c_m\, e_{m,k}\, y_k^{e_{m,k}-1} \prod_{i\neq k} y_i^{e_{m,i}}$. Each monomial generates a separate integral term with:

- Coefficient: $B_j \cdot c_m \cdot e_{m,k}$
- New exponents: $\alpha_{t,k} = a_k + e_{m,k}$ (convergent since $e_{m,k} \geq 1$), $\alpha_{t,i} = a_i + e_{m,i}$ for $i \neq k$
- Polynomial factors: $Q_j^{B_j-1} \prod_{l\neq j} Q_l^{B_l}$

Step 3: ε-expansion before flattening. Each term's exponents $\alpha_{t,i}(\varepsilon)$ and polynomial exponents $\tilde{B}_j(\varepsilon)$ are expanded in ε. At order ε^0: the base integrand (no log insertions). At order ε^1: the base integrand times log-insertion sum $\sum_i(\alpha_{t,i}^{(1)}/\alpha_{t,i}^{(0)})\log y_i' + \sum_j \tilde{B}_j^{(1)}\log Q_j$.

Step 4: Per-term flattening. Each term gets its own flattening $y_i \to (y_i')^{1/\alpha_{t,i}^{(0)}}$ since different terms have different zeroth-order exponents.

Step 5: Iterative application for nested divergences. When multiple variables are divergent, apply IBP iteratively. After resolving y_{k_1}, check each resulting term for remaining divergences and apply IBP to y_{k_2} as needed. With d nested divergences, poles up to $1/\varepsilon^d$ are produced.

7.9 Treatment of Complex Exponents

The exponents A_i (monomial) and B_j (polynomial) may be complex-valued. The pipeline handles this by cleanly separating the combinatorial steps—which depend only on real integer data—from the analytic steps, which carry the full complex values throughout.

What is real/combinatorial. The following operations depend only on the Newton polytope (the set of exponent vectors appearing in the polynomials) and are therefore insensitive to the complex parts of A_i and B_j:

- Newton polytope construction and convex hull computation.
- Normal fan and simplicial triangulation (ray vectors ρ_i are integer).
- The ray matrix $M = -\text{Transpose}[\rho_1, \ldots, \rho_n]$ and its determinant.
- Tropical factoring: the minimum exponents $d_{k,j} = \min_m\{e_{m,j}\}$ are determined by the transformed polynomial exponent vectors, which are integer linear combinations of the original (integer) exponent vectors. These do not depend on A_i or B_j at all.

What is complex. The following quantities use the full complex values of A_i and B_j, and are computed symbolically in Mathematica before being passed to C++:

- **Effective exponents** (line 227 of `tropical_eval.wl`):

$$a_{\text{eff},i} = \text{rawA}_i + \sum_k B_k \cdot d_{k,i} \;\in\; \mathbb{C} \tag{35}$$

  Here $\text{rawA}_i = -1 + \sum_k (A_k + 1) M_{ki}$, which is complex when the A_k are. The B_k contribute their complex parts through the $B_k \cdot d_{k,i}$ term.

- **Flattening** (line 286): the exponent division $e_{m,i}/a_{\text{eff},i}$ uses the full complex $a_{\text{eff},i}$, producing complex-valued flattened exponents. In the C++ code these appear inside the log-exp evaluation:

$$y_i^{e/a_{\text{eff},i}} = \exp\big((e/a_{\text{eff},i}) \cdot \log y_i\big) \tag{36}$$

  which is well-defined for $y_i \in (0, 1]$ and complex exponent ratio.

- **Prefactor** (line 293):

$$\text{prefactor} = \frac{|\det(M)|}{\prod_i a_{\text{eff},i}} \;\in\; \mathbb{C} \tag{37}$$

- **Convergence check** (line 246): only $\mathrm{Re}(a_{\mathrm{eff},i})$ is tested. A sector is convergent if and only if $\mathrm{Re}(a_{\mathrm{eff},i}) > 0$ for all i (at $\varepsilon = 0$ when a regulator is present). The imaginary part does not affect convergence.
- **Divergent sector processing**: In `ProcessDivergentSector`, the G_0 flattening (line 680) divides by $a_0[\texttt{ndVars}]$ (the $\varepsilon = 0$ evaluation of the full complex effective exponents), and the remainder flattening (line 719) divides by $a_0[i]$. Both carry complex values.

Summary. The computational strategy is:

1. Perform the tropical decomposition using only the combinatorial structure of the polynomials (integer exponent vectors). This is purely real.
2. Compute $a_{\mathrm{eff},i}$ and the flattening transform using the full complex A_i and B_j. All downstream quantities (flattened exponents, prefactors, C++ integrands) are complex-valued.
3. In C++, all arithmetic is `std::complex<double>`. The log-exp evaluation $\exp(\alpha \log y)$ with complex α and real $y > 0$ is numerically stable and branch-cut free.

8 Dependencies

8.1 External Software

Polymake (https://polymake.org)
C++ library with Perl interface for polyhedral computation. Used for convex hulls, vertex enumeration, fan computation, and simplicial triangulation. Must be installed and accessible on `PATH`, or in one of: `/opt/homebrew`, `/usr/local`, `/usr`.

g++ (GCC C++ compiler)
C++17 standard required (`-std=c++17`). OpenMP support recommended (`-fopenmp`) for parallel kinematic evaluation. If OpenMP linking fails (common on macOS with clang), the code auto-retries without `-fopenmp`.

Wolfram Mathematica (version 12.0+)
The symbolic computation engine. Required for running all `.wl` files. `wolframscript` used for command-line execution.

8.2 Internal Dependencies

- `tropical_fan.wl` must be loaded before `tropical_eval.wl` (the latter calls `Get["tropical_fan.wl"]` internally).
- No external Wolfram packages required beyond the core installation.

9 Usage and Workflow

9.1 Minimal Convergent Integral

```
(* Load the package *)
Get["tropical_eval.wl"];

(* Define the integrand *)
poly = 1 + 2*x[1]^2 + x[2]^2 + x[1]*x[2]^2;
vars = {x[1], x[2]};
```

```
spec = <|
  "Polynomials"       -> {poly},
  "MonomialExponents" -> {0, 0},
  "PolynomialExponents" -> {-2},
  "Variables"         -> vars,
  "KinematicSymbols"  -> {},
  "RegulatorSymbol"   -> None
|>;

(* Compute the fan *)
verts  = PolytopeVertices[poly^(-1), vars];
fanData = ComputeDecomposition[verts];

(* Validate against NIntegrate *)
vr = ValidateDecomposition[spec, fanData, {}, 3];
Print["Relative error: ", vr["RelativeError"]];

(* Full MC evaluation *)
results = EvaluateTropicalMC[spec, fanData, {}];
```

9.2 Kinematic Scan (Many Evaluation Points)

```
Get["tropical_eval.wl"];

(* Polynomial with kinematic-dependent coefficients *)
poly = c0 + c1*x[1]^2 + c2*x[2]^2;

spec = <|
  "Polynomials"       -> {poly},
  "MonomialExponents" -> {0, 0},
  "PolynomialExponents" -> {-2},
  "Variables"         -> {x[1], x[2]},
  "KinematicSymbols"  -> {c0, c1, c2},
  "RegulatorSymbol"   -> None
|>;

(* Fan uses unit coefficients -- structure only *)
proxyPoly = 1 + x[1]^2 + x[2]^2;
verts = PolytopeVertices[proxyPoly^(-1), vars];
fanData = ComputeDecomposition[verts];

(* Define 7200 kinematic points *)
kinPoints = Table[{1.0, 0.5+0.01*i, 1.0+0.005*i}, {i, 7200}];

(* Evaluate: one compilation, parallel execution *)
results = EvaluateTropicalMC[spec, fanData, kinPoints];
```

Note: The fan decomposition depends only on the Newton polytope structure (which monomials appear), not on numerical coefficients. The same decomposition works for all kinematic points, requiring only one C++ compilation.

9.3 Divergent Integral with Regulator

```
Get["tropical_eval.wl"];
```

```
spec = <|
  "Polynomials"       -> {1 + x[1] + x[2] + x[1]*x[2]},
  "MonomialExponents" -> {2*eps - 1, 0},
  "PolynomialExponents" -> {-2},
  "Variables"         -> {x[1], x[2]},
  "KinematicSymbols"  -> {},
  "RegulatorSymbol"   -> eps
|>;

verts = PolytopeVertices[
  (1 + x[1] + x[2] + x[1]*x[2])^(-1), {x[1], x[2]}];
fanData = ComputeDecomposition[verts];

(* Evaluate at eps = 0.01 *)
results = EvaluateTropicalMC[spec, fanData, {},
  EpsilonValue -> 0.01];
```

The result includes: analytic pole coefficient/ε, numerical finite part (G_1/c_k+ Remainder), and total with error bars.

9.4 Command-Line Execution

```
# Quick smoke test (Example 1 only)
wolframscript -file EXAMPLES/test_quick.wl

# C++ code generation and execution test
wolframscript -file EXAMPLES/test_cpp.wl

# Divergent integral tests
wolframscript -file EXAMPLES/test_divergent.wl

# Full example suite (14 examples + 18 tests)
wolframscript -file EXAMPLES/tropical_eval_examples.wl

# Fan computation examples
wolframscript -file EXAMPLES/tropical_fan_examples.wl
```

10 Examples Directory

The `EXAMPLES/` directory contains worked examples and validation tests.

`tropical_eval_examples.wl` — 14 Worked Examples

Example	Description
1	Basic 2D convergent integral
2	Complex exponents (A_i and B_j with imaginary parts)
3	Kinematic-dependent integral (coefficients are parameters)
4	Tropical factoring step-by-step walkthrough
5	`MmaToC` expression converter demonstration
6	C++ code generation and execution (full pipeline)
7	C++ with kinematic scan (multiple evaluation points)
8	`ParsePolynomial` utility function
9	Multiple polynomials (2 factors in denominator)
10	Non-trivial numerator (1 polynomial, $A_i \neq 0$)
11	Non-trivial numerator (2 polynomials)
12	Numerator with kinematic-dependent coefficients
13	3D convergent integral (6 rays, 8 sectors)
14	4D convergent integral (8 rays, 14 sectors)

Other Example Files

`tropical_fan_examples.wl` — Fan decomposition examples for various polytopes.

`test_quick.wl` — Minimal test running only Example 1.

`test_cpp.wl` — Tests C++ code generation, compilation, and execution.

`test_divergent.wl` — Tests the divergence regulation pipeline.

`prompt_tropical_mc.txt` — Original development specification.

11 Validation and Testing

11.1 22-Test Suite (in `tropical_eval_examples.wl`)

Convergent Tests (1–7)

Test 1: 2D real exponents. Benchmarked against exact result via polynomial transformation.

Test 2: 2D complex exponents. Cross-checked against `NIntegrate`.

Test 3: 1D divergent (pole extraction). Verifies analytic pole coefficient.

Test 4: 2D divergent. Checks G_0, G_1, and Remainder separation.

Test 5: 100-point kinematic scan with complex exponents.

Test 6: Large coefficient range (10^{-4} to 10^{8}).

Test 7: 3D and 4D integrals. 3D: 6 rays, 8 sectors; 4D: 8 rays, 14 sectors.

Divergent Tests (8–18)

Test 8: Full divergent pipeline (`ProcessSector` → `ProcessDivergentSector` → `ValidateSubtraction`).

Test 9: Multi-ε convergence.

Test 10: Divergent variable permutation (not always y_1).

Test 11: ε-dependent polynomial exponents ($B_j = B_j(\varepsilon)$).

Test 12: Full sector sum (convergent + divergent).

Test 13: C++ codegen for divergent sectors.

Test 14: Pole coefficient accuracy (analytic vs. numerical).

Test 15: G_1 log-insertion structural check.

Test 16: Multiple-polynomial divergent.

Test 17: 3D divergent integral.

Test 18: Complex polynomial exponents with $1/\varepsilon$ pole.

IBP Tests (19–22)

Test 19: IBP symbolic decomposition. Verifies `IBPReduceSector` produces the correct number of terms (matching the count of monomials with $e_{m,k} > 0$) and that all terms have convergent exponents at $\varepsilon = 0$.

Test 20: IBP cross-check via `ValidateIBP`. Verifies that $(1/a_k)[B_{\text{boundary}} - \sum_t \text{coeff}_t I_t]$ reproduces the original sector integral at finite ε (threshold: 5%).

Test 21: IBP term count with multi-monomial polynomial (5 monomials). Verifies term count matches per sector.

Test 22: Nested divergence (two divergent variables). Constructs a 2D integral with both x_1 and x_2 divergent ($A_i = 2\varepsilon - 1$ for both). Verifies iterative IBP resolves all divergences: $1 \to 2 \to 3$ terms.

11.2 FIESTA Cross-Checks (`CC/` Directory)

The `CC/` directory contains cross-validation against **FIESTA**, an independent sector decomposition code used in the physics community.

Simple tests: $\pi/8$ integral (convergent, agreement to 6 decimal places), Gamma function integral (divergent, pole matches exactly), 2D with kinematics.

Full 5D cosmological integral (bubblepp presector 1):

- 36 convergent sectors + 14 divergent sectors
- All divergent sectors have $c_k = 2/7$
- Convergence study: $N = 50{,}000$ to $10{,}000{,}000$ samples
- Error scales as $1/\sqrt{N}$
- Results match FIESTA to within 2σ of MC error
- Verified via GL(1)/Cheng–Wu gauge transformation

12 Limitations and Caveats

1. **Divergence handling.** The tropical subtraction scheme (`ProcessDivergentSector`) handles at most one divergent variable per sector. For nested divergences (multiple divergent variables), use the IBP-based pipeline (`IBPProcessSector` / `EvaluateTropicalMCIBP`), which handles arbitrary nesting depth. The number of IBP terms grows combinatorially with the nesting depth ($\mathcal{O}(M^d)$ for d divergent variables and M total monomials).

2. **Positive real polynomial coefficients.** All polynomial coefficients must be real and positive, ensuring $\log(P_j)$ is real-valued. Complex integrands arise only from the exponents A_i and B_j.

3. **Monte Carlo convergence.** Accuracy scales as $1/\sqrt{N}$. High-variance sectors may require $> 10^6$ samples.

4. **Polymake dependency.** External tool, not bundled. Must be installed separately.

5. **Newton polytope stability.** If kinematic parameters can cause coefficients to vanish, the Newton polytope changes and the fan must be recomputed. The code assumes fixed polytope structure across all kinematic points.

6. **C++ compiler requirements.** `g++` with C++17 required. OpenMP optional but recommended. macOS clang may require fallback to single-threaded mode.

7. **Double precision.** All C++ computation uses 64-bit doubles. Higher precision would require additional libraries.

8. **Memory scaling.** Results scale linearly with the number of kinematic points. Very large scans ($\gg$ 10,000 points) may require attention to memory.

13 Mathematical Properties and Why It Works

Why tropical decomposition works. As $\mathbf{x} \to 0$ in a direction $\mathbf{w}$, the polynomial $P(\mathbf{x})$ is dominated by the monomial whose exponent vector minimizes the dot product with $\mathbf{w}$. The tropical fan exactly partitions $\mathbb{R}^n$ into regions where each monomial dominates. Within each sector, the dominant monomial is factored out, leaving a polynomial $Q(\mathbf{y})$ bounded away from zero.

Why flattening is important. Without flattening, the integrand contains factors $y_i^{a_i-1}$ which can be highly peaked near $y_i = 0$ when a_i is small. Uniform MC sampling wastes most samples away from the peak, leading to enormous variance. Flattening absorbs the y^{a-1} singularity into the integration measure, converting the integrand to $\mathcal{O}(1)$ and dramatically reducing MC variance.

Why log-exp evaluation is stable. For $y \in (0, 1]$, $\log(y)$ varies smoothly in $(-\infty, 0]$. The exponential $\exp(\alpha \log y) = y^\alpha$ handles fractional and complex α without branch cuts or sign ambiguities. As $y \to 0^+$, the function decays or grows depending on $\mathrm{Re}(\alpha)$, but always smoothly.

Why tropical factoring is exact. The identity $P(\mathbf{y}) = \mathbf{y}^d \cdot Q(\mathbf{y})$ is algebraic, not a numerical approximation. Every monomial in Q has non-negative exponents by construction. The constant term in Q ensures $Q \geq \text{const} > 0$ on $[0, 1]^n$. No numerical error is introduced.

Why the pole extraction is correct. The $1/(c_k\varepsilon)$ pole arises from:

$$\int_0^1 y_k^{c_k\varepsilon-1}\,dy_k = \frac{1}{c_k\varepsilon} \tag{38}$$

This is exact. The subtraction $f(\mathbf{y}) - f(\mathbf{y}|_{y_k=0})$ vanishes at $y_k = 0$, so the difference integral is convergent for $\varepsilon = 0$. The G_1 correction captures the $\mathcal{O}(\varepsilon^0)$ contribution from the ε-expansion. Together, these give the complete Laurent expansion to order $\mathcal{O}(\varepsilon^0)$.

Why the IBP pole extraction is correct. The IBP identity on $[0, 1]$ gives:

$$a_k \int_0^1 y_k^{a_k-1} f\,dy_k = f\big|_{y_k=1} - \int_0^1 y_k^{a_k}\partial_{y_k} f\,dy_k \tag{39}$$

The boundary at $y_k = 0$ vanishes for $\mathrm{Re}(a_k) > 0$ (i.e., for $\varepsilon > 0$). The boundary at $y_k = 1$ is a well-defined $(n-1)$-dimensional integral. Dividing by $a_k = c_k\varepsilon + \cdots$ produces the $1/\varepsilon$ pole.

Each IBP term has effective exponent $\alpha_{t,k}^{(0)} = e_{m,k} \geq 1$ for the previously-divergent variable, ensuring convergence. The monomial decomposition avoids introducing $(\partial_{y_k} Q_j)/Q_j$ ratios in the C++ integrands—each term evaluates only standard polynomial-power expressions.

For nested divergences, successive IBP steps produce higher-order poles $1/\varepsilon^d$. The identity $B^{(0)} - \sum_t \mathrm{coeff}_t^{(0)} I_{t,0} = G_0$ connects the IBP result to the subtraction scheme's pole coefficient, providing a cross-check.